

OpenVLA-LIBERO 可视化评测实验手册

1. 实验目的

本实验用于在 LIBERO Benchmark 上运行 OpenVLA，并观察机器人策略执行过程。

实验流程：

```
graph TD
    A[OpenVLA模型] --> B[输入图像 + 语言指令]
    B --> C[预测机器人动作]
    C --> D[LIBERO MuJoCo仿真环境]
    D --> E[机器人执行任务]
    E --> F[统计Success Rate]
```

2. 环境配置

推荐环境：

软件 版本

Ubuntu 22.04

Python 3.10

CUDA 12.x

PyTorch 2.2

创建环境：

```
conda create -n openvla python=3.10 -y
conda activate openvla
conda install pytorch torchvision torchaudio pytorch-cuda=12.4 -c pytorch -c
nvidia -y # UPDATE ME!
```

安装 OpenVLA：

```
git clone https://github.com/openvla/openvla
cd openvla
pip install -e .
```

安装 FlashAttention：

```
pip install packaging ninja
ninja --version; echo $? # verify Ninja --> should return exit code "0"
pip install flash-attn==2.5.5 --no-build-isolation
```

若flash-attn编译过慢或者有问题可放弃flash-attn加速，并对
openvla/experiments/robot/openvla_utils.py进行以下修改

```
def get_vla(cfg):
    """Loads and returns a VLA model from checkpoint."""
    # Load VLA checkpoint.
    print("[*] Instantiating Pretrained VLA model")
    print("[*] Loading in BF16 with Flash-Attention Enabled")
    print("[*] Loading in BF16 with PyTorch SDPA")

    # Register OpenVLA model to HF Auto Classes (not needed if the model is on HF Hub)
    AutoConfig.register("openvla", OpenVLAConfig)
    AutoImageProcessor.register(OpenVLAConfig, PrismaticImageProcessor)
    AutoProcessor.register(OpenVLAConfig, PrismaticProcessor)
    AutoModelForVision2Seq.register(OpenVLAConfig, OpenVLAForActionPrediction)

    vla = AutoModelForVision2Seq.from_pretrained(
        cfg.pretrained_checkpoint,
        attn_implementation="flash_attention_2",
        attn_implementation="sdpa",
        torch_dtype=torch.bfloat16,
        load_in_8bit=cfg.load_in_8bit,
        load_in_4bit=cfg.load_in_4bit,
        low_cpu_mem_usage=True,
        trust_remote_code=True,
    )
```

3. LIBERO安装

下载 LIBERO:

```
git clone https://github.com/LifeLong-Robot-Learning/LIBERO.git

cd LIBERO

pip install -e .
```

安装评测依赖:

```
cd openvla

pip install -r experiments/robot/libero/libero_requirements.txt
```

4. 下载模型

OpenVLA 官方提供 LIBERO 微调模型:

- libero_spatial
- libero_object

- libero_goal
- libero_10

首次运行时会自动从 HuggingFace 下载。

建议使用autodl自带的学术资源加速，终端中输入

```
source /etc/network_turbo
```

安装huggingface_hub

```
pip install huggingface--hub
```

使用镜像网站下载指定模型到指定位置（以spatial为例）

```
export HF_ENDPOINT=https://hf-mirror.com
hf download openvla/openvla-7b-finetuned-libero-spatial --local-dir LOCAL_DIR(替换
下载到的位置)
```

下载缓慢时可以尝试中断重连，不用担心进度清零，会保存到.cache里

5. LIBERO评测

Spatial任务

```
python experiments/robot/libero/run_libero_eval.py \
--model_family openvla \
--pretrained_checkpoint openvla/openvla-7b-finetuned-libero-spatial \
--task_suite_name libero_spatial \
--center_crop True
```

Object任务

```
python experiments/robot/libero/run_libero_eval.py \
--model_family openvla \
--pretrained_checkpoint openvla/openvla-7b-finetuned-libero-object \
--task_suite_name libero_object \
--center_crop True
```

Goal任务

```
python experiments/robot/libero/run_libero_eval.py \
--model_family openvla \
--pretrained_checkpoint openvla/openvla-7b-finetuned-libero-goal \
--task_suite_name libero_goal \
--center_crop True
```

Long任务

```
python experiments/robot/libero/run_libero_eval.py \
--model_family openvla \
--pretrained_checkpoint openvla/openvla-7b-finetuned-libero-10 \
--task_suite_name libero_10 \
--center_crop True
```

6. LIBERO Benchmark简介

Benchmark 主要能力

Spatial 空间关系理解
Object 物体泛化能力
Goal 语言目标理解
Long 长时间连续操作能力

7. 可视化结果

运行后可以观察：

```
RGB Camera
  ↓
OpenVLA预测动作
  ↓
机械臂执行
  ↓
MuJoCo物理仿真
  ↓
任务成功/失败
```

输出通常包括：

```
rollout视频
evaluation log
success rate
```

输出的视频将会放在rollouts里

8. 常见问题

libero无法导入

检查：

```
pip install -e LIBERO路径
```

MuJoCo EGL错误

检查 NVIDIA 驱动和 CUDA 环境。

FlashAttention安装失败

确认 PyTorch、CUDA版本匹配。

9. 扩展实验 (有时间选做)

替换：

```
--pretrained_checkpoint
```

即可测试自己微调的OpenVLA 模型

1、下载原始模型权重

建议使用autodl自带的学术资源加速，终端中输入

```
source /etc/network_turbo
```

安装huggingface_hub

```
pip install huggingface--hub
```

使用镜像网站下载指定模型到指定位置

```
export HF_ENDPOINT=https://hf-mirror.com
hf download openvla/openvla-7b --local-dir LOCAL_DIR(替换下载到的位置)
```

2、git拉取代码

```
git clone https://github.com/niejnan/OpenVLA.git
```

3、开始训练

```
python OpenVLA/finetune.py
```

更改对应位置配置

```
class FinetuneConfig:
    vla_path: str = "/root/autodl-tmp/openvla-7b" # 本地的OpenVLA原始模型配置

    # RLDS 数据集目录
    data_root_dir: Path = Path("/root/autodl-tmp/libero") #数据集下载位置
    dataset_name: str = "libero_spatial_no_noops" #你所选的评测类型
```

```

# 结果保存目录
run_root_dir: Path = Path("runs")                                #结果保存目录

# Fine-tuning Parameters
batch_size: int = 8
learning_rate: float = 5e-4
use_lora: bool = True                                           # whether to
use LoRA fine-tuning
lora_rank: int = 32                                             # Rank of
LoRA weight matrix
lora_dropout: float = 0.05                                     # Dropout
applied to LoRA weights
use_quantization: bool = False                                 # whether to
4-bit quantize VLA for LoRA fine-tuning

# =>

CAUTION: Reduces memory but hurts performance

# 微调的最大时间步数（类似epoch）
max_steps: int = 200_000

# 保存检查点的间隔
save_steps: int = 2500

# 梯度累积
"""
累积梯度： 当你设置 grad_accumulation_steps 大于 1 时，模型不会在每个 batch 之后立即
更新参数，而是累计多个 batch 的梯度。只有累计了指定步数的梯度后，才会进行一次参数更新。
模拟大批量训练： 如果显存有限，无法使用大 batch size，梯度累积可以在不增加显存占用的情况
下，模拟大 batch size 的效果，从而稳定训练。
实现方式： 例如，如果 grad_accumulation_steps=4，那么在每四个 batch 后调用一次
optimizer.step() 更新模型参数，并在每次更新后清空累积的梯度。
这种方式可以在显存有限的情况下，通过梯度累积实现更稳定的训练效果。
"""
grad_accumulation_steps: int = 2

# 图像增强
image_aug: bool = True                                         # whether to
train with image augmentations

# 数据加载器用于打乱的缓冲区大小（如果内存不足可以减少）
shuffle_buffer_size: int = 100_000                             # DataLoader
shuffle buffer size (can reduce if OOM)

# 加入运行 id 的额外注释
run_id_note: Optional[str] = None

```

4、运行

`OpenVLA/config.py` 中的：

1. `pretrained_checkpoint` 字段的值修改为----模型权重所在的文件夹
2. `lora_checkpoint` 字段的值修改为---- LoRA 权重所在的文件夹

```
class Config:
    model_family: str = "openvla"
    # Pretrained checkpoint path
    pretrained_checkpoint: Union[str, Path] = "/mnt/workspace/openvla-7b-
finetuned"
    .....
    # lora 权重路径
    lora_checkpoint: str = "your_lora_checkpoint"
```

3. 运行仿真

```
python OpenVLA/eval.py
```

在同一 LIBERO Benchmark 下比较性能。

5、可能遇见问题

protobuf版本问题

```
pip install --force-reinstall protobuf==6.31.1
```

缺少libegl1库等

```
python -m pip install -e . --config-settings editable_mode=compat
```

```
apt update
apt-get install -y libegl1 libglvnd0
```